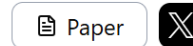


AI in the News

- https://www.agidefinition.ai/?utm_source=substack&utm_medium=email



A Definition of AGI



Dan Hendrycks¹, Dawn Song², Christian Szegedy³, Honglak Lee⁴, Yarin Gal⁵
Sharon Li⁶, Andy Zou^{1,7,8}, Lionel Levine⁹, Bo Han¹⁰, Jie Fu¹¹, Ziwei Liu¹², Jinwoo Shin¹³
Kimin Lee¹³, Mantas Mazeika¹, Long Phan¹, George Ingebrechtsen¹, Adam Khoja¹
Cihang Xie¹⁴, Olawale Salaudeen¹⁵, Matthias Hein¹⁶, Kevin Zhao¹⁷, Alex Pan²
David Duvenaud^{18,19}, Bo Li²⁰, Steve Omohundro²¹, Gabriel Alfour²², Max Tegmark¹⁵
Kevin McGrew²³, Gary Marcus²⁴, Jaan Tallinn²⁵, Eric Schmidt¹⁵, Yoshua Bengio^{26,27}

¹Center for AI Safety

⁴University of Michigan

⁷Gray Swan AI

¹⁰Hong Kong Baptist University

¹³KAIST

¹⁶University of Tübingen

¹⁹Vector Institute

²University of California, Berkeley

⁵University of Oxford

⁸Carnegie Mellon University

¹¹HKUST

¹⁴University of California, Santa Cruz

¹⁷University of Washington

²⁰University of Chicago

³Morph Labs

⁶University of Wisconsin–Madison

⁹Cornell University

¹²Nanyang Technological University

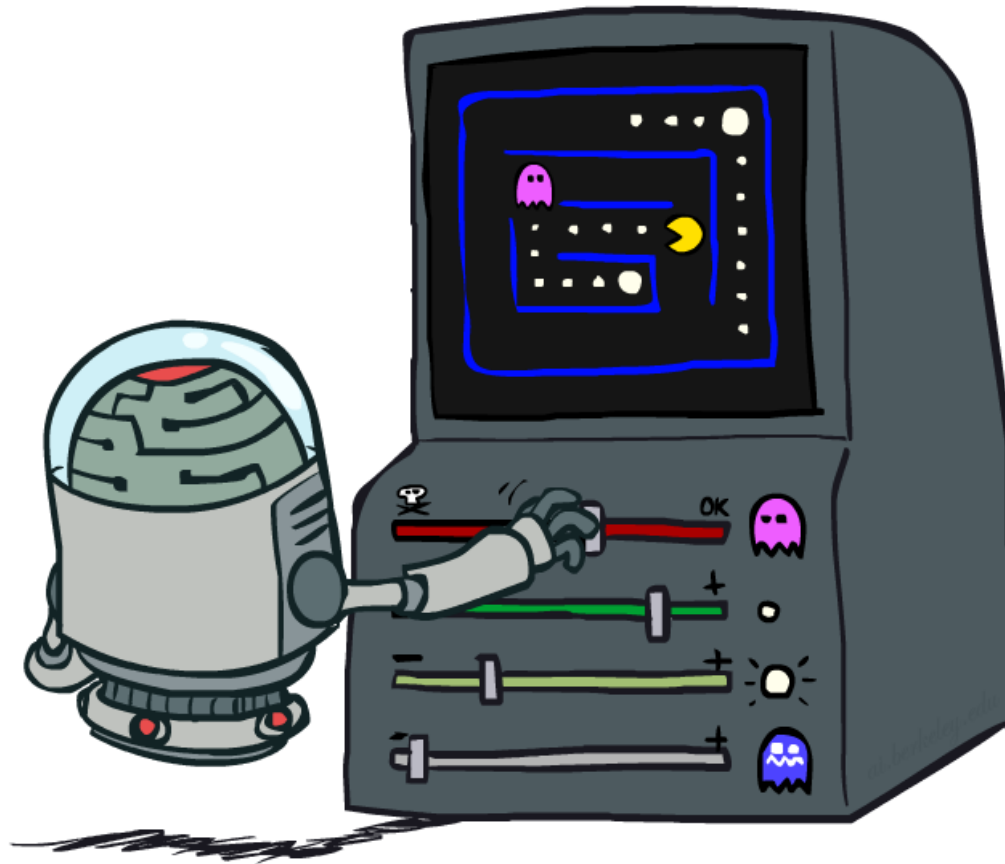
¹⁵Massachusetts Institute of Technology

¹⁸University of Toronto

²¹Beneficial AI Research

CS 6300: Artificial Intelligence

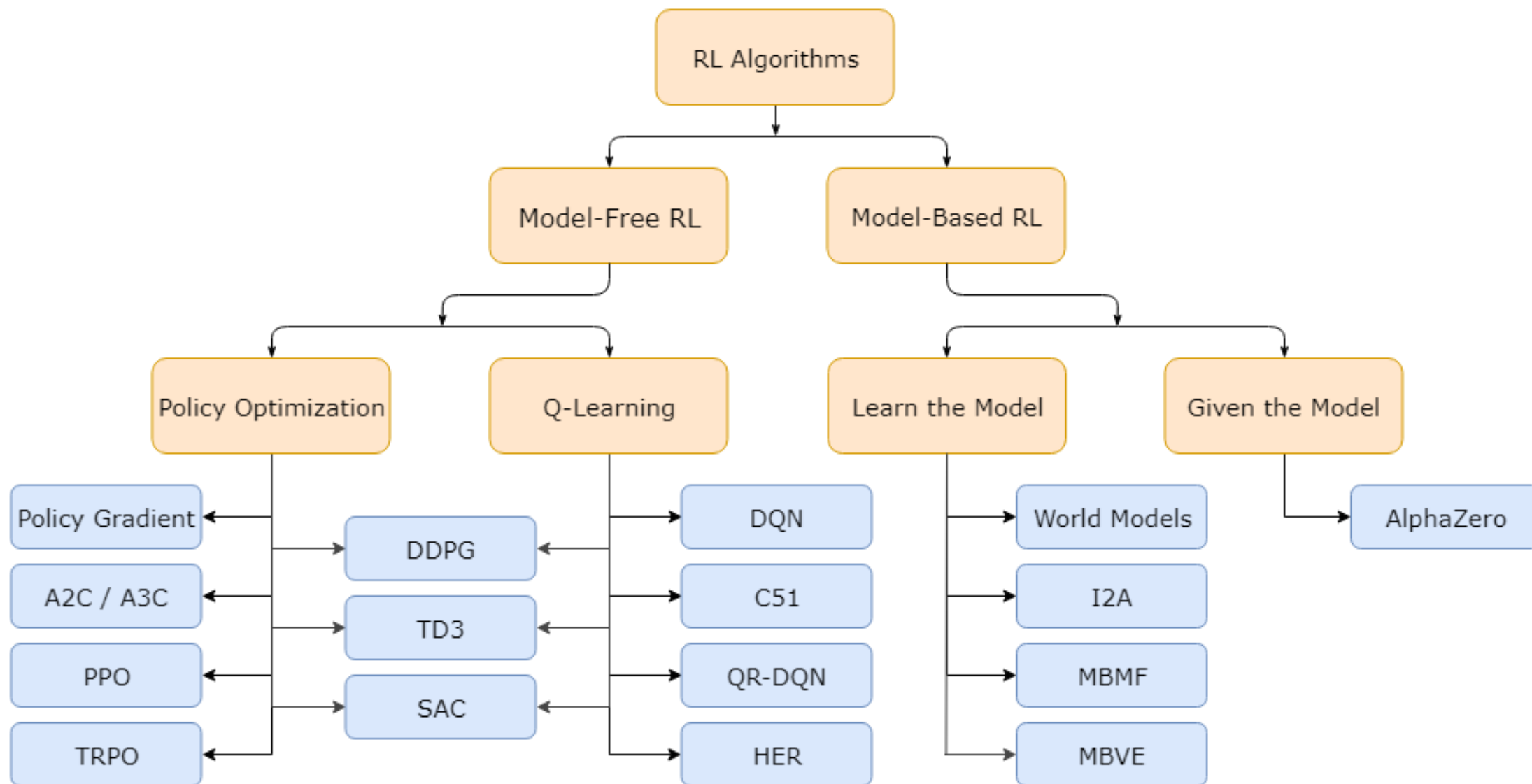
Reinforcement Learning III: Policy Gradients

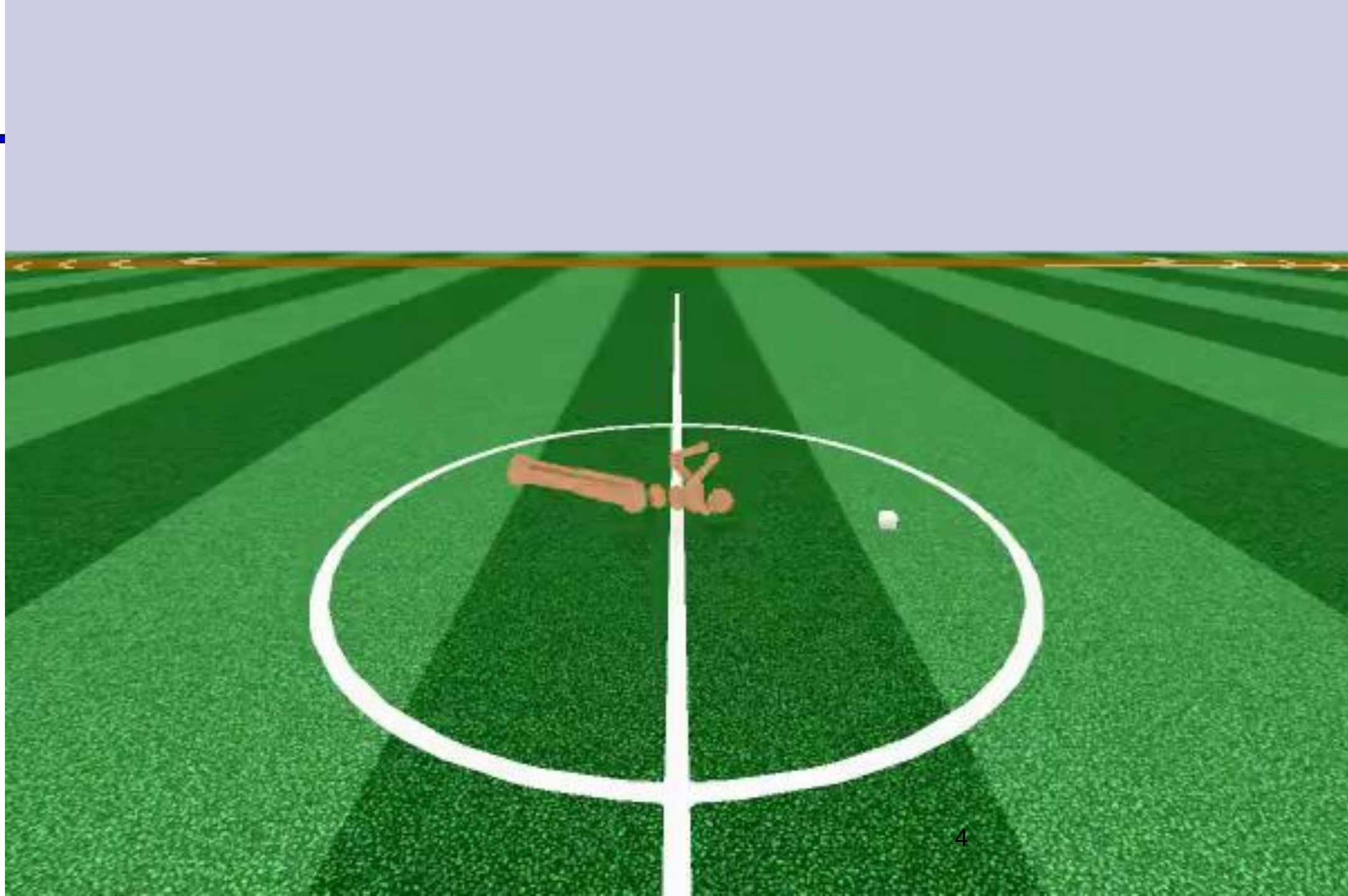


Instructor: Daniel Brown --- University of Utah

[Based on slides created by Dan Klein and Pieter Abbeel <http://ai.berkeley.edu>.]

Rough Taxonomy of RL Algorithms







What is the goal of RL?

- Find a policy that maximizes expected utility (discounted cumulative rewards)

$$\pi^* = \underset{\pi}{\operatorname{argmax}} E_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s, \pi(s), s') \right]$$

Two approaches to model-free RL

■ Learn Q-values

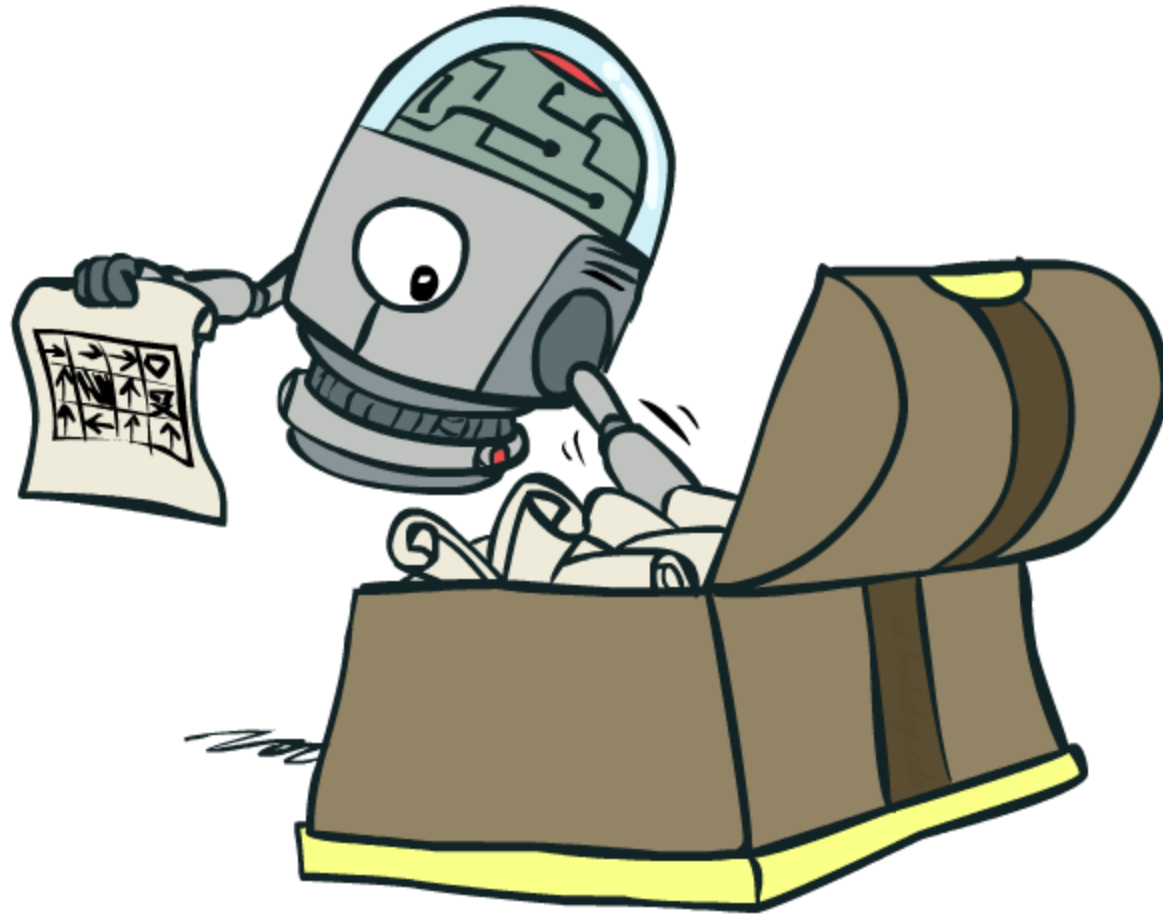
- Trains Q-values to be consistent. Not directly optimizing for performance.
- Use an objective based on the Bellman Equation

$$Q_{k+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma \max_{a'} Q_k(s', a') \right]$$

■ Learn Policy Directly

- Have a parameterized policy π_{θ}
- Update the parameters θ to optimize performance of policy.

Policy Search



Policy Search

- Problem: often the feature-based policies that work well (win games, maximize utilities) aren't the ones that approximate V / Q best
 - E.g. your value functions from project 2 were probably horrible estimates of future rewards, but they still produced good decisions
 - Q-learning's priority: get Q-values close (modeling)
 - Action selection priority: get ordering of Q-values right (prediction)
 - We'll see this distinction between modeling and prediction again later in the course
- Solution: learn policies that maximize rewards, not the values that predict them
- Policy search: start with an ok solution (e.g. Q-learning) then fine-tune by hill climbing on feature weights

Policy Search

- Simplest policy search:
 - Start with an initial linear value function or Q-function
 - Nudge each feature weight up and down and see if your policy is better than before
- Problems:
 - How do we tell the policy got better?
 - Need to run many sample episodes!
 - If there are a lot of features, this can be impractical
- Better methods exploit lookahead structure, sample wisely, change multiple parameters...

Policy Search



Preliminaries

- Trajectory (rollout, episode) $\tau = (s_0, a_0, s_1, a_1, \dots)$
 - $s_0 \sim \rho_0(\cdot), \quad s_{t+1} \sim P(\cdot | s_t, a_t)$
- Rewards $r_t = R(s_t, a_t, s_{t+1})$
- Finite-horizon undiscounted return of a trajectory

$$R(\tau) = \sum_{t=0}^T r_t$$

- Actions are sampled from a parameterized policy π_θ
 $a_t \sim \pi_\theta(\cdot | s_t)$

Preliminaries

- Probability of a trajectory (rollout, episode) $\tau = (s_0, a_0, s_1, a_1, \dots)$

$$P(\tau|\pi) = \rho_0(s_0) \prod_{t=0}^{T-1} P(s_{t+1}|s_t, a_t) \pi_{\theta}(a_t|s_t)$$

- Expected Return of a policy $J(\pi)$

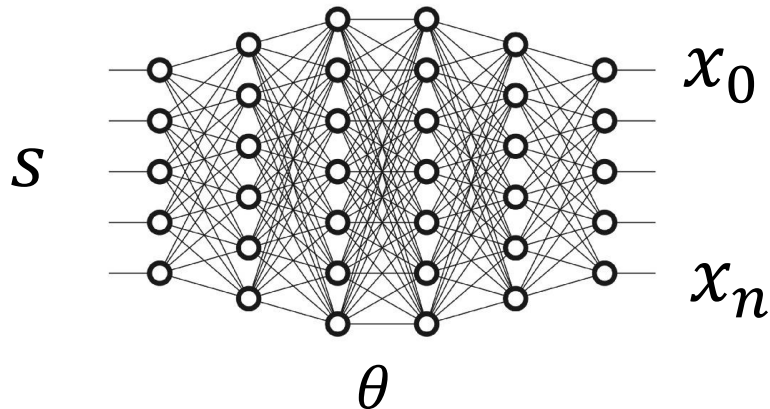
$$J(\pi) = \sum_{\tau} P(\tau|\pi) R(\tau) = E_{\tau \sim \pi}[R(\tau)]$$

- Goal of RL: Solve the following optimization problem

$$\pi^* = \operatorname{argmax}_{\pi} J(\pi)$$

How should we parameterize our policy?

- We need to be able to do two things:
 - Sample actions $a_t \sim \pi_\theta(\cdot | s_t)$
 - Compute log probabilities $\log \pi_\theta(a_t | s_t)$
- Categorical (classifier over discrete actions)
 - Typically, you output a value x_i for each action (class) and then the probability is given by a softmax equation



$$\pi_\theta(a_i | s) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$$

How should we parameterize our policy?

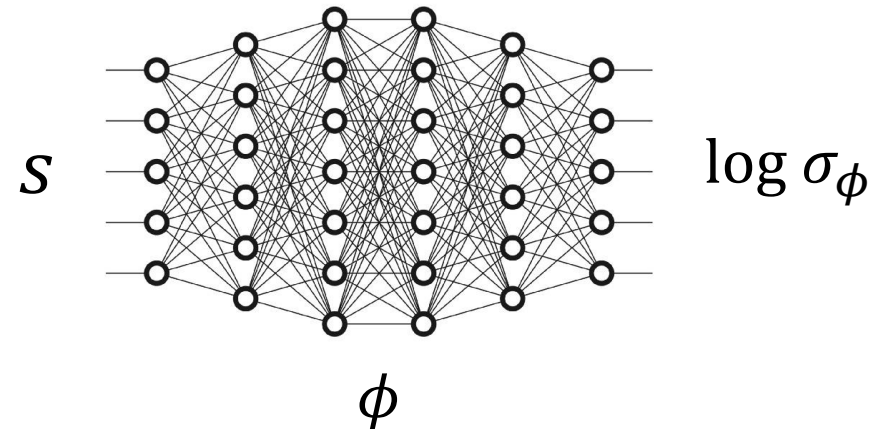
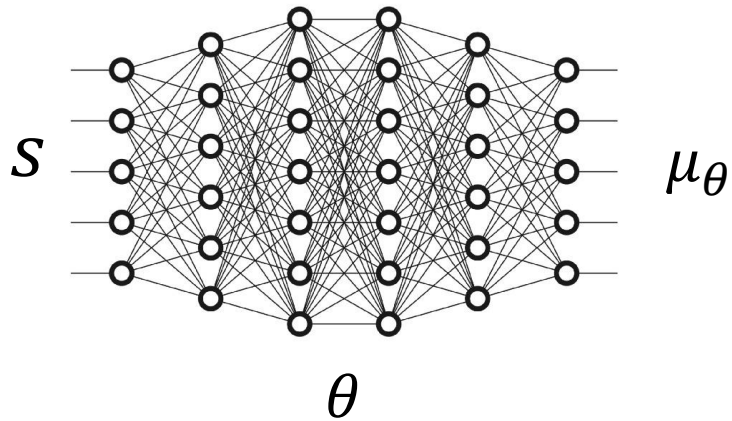
- Diagonal Gaussian (distribution over continuous actions)

$$a \sim N(\mu, \Sigma)$$

where Σ has non-zero elements only on the diagonal.

Thus, an action can be sampled as

$$a = \mu_{\theta}(s) + \sigma_{\phi}(s) \cdot z, \quad z \sim N(0, I)$$



Goal: Update Policy via Gradient Ascent

- We have a parameterized policy and we want to update it so that it maximizes the expected return.
- We want to find the gradient of the return with respect to the policy parameters and step in that direction.

$$\theta_{k+1} \leftarrow \theta_k + \alpha \nabla_{\theta} J(\pi_{\theta}) \Big|_{\theta_k}$$

Policy gradient

Fact #1

- Probability of a trajectory:

- The probability of a trajectory $\tau = (s_0, a_0, \dots, s_{T+1})$ given that actions come from π_θ is

$$P(\tau|\pi) = \rho_0(s_0) \prod_{t=0}^{T-1} P(s_{t+1}|s_t, a_t) \pi_\theta(a_t|s_t)$$

Fact #2

- Log-probability of a trajectory:

- The log-probability of a trajectory $\tau = (s_0, a_0, \dots, s_{T+1})$ given that actions come from π_θ is

$$\begin{aligned}\log P(\tau|\pi) &= \log \left(\rho_0(s_0) \prod_{t=0}^T P(s_{t+1}|s_t, a_t) \pi_\theta(a_t|s_t) \right) \\ &= \log \rho_0(s_0) \\ &\quad + \sum_{t=0}^T (\log P(s_{t+1}|s_t, a_t) + \log \pi_\theta(a_t|s_t))\end{aligned}$$

Fact #3

- Grad-Log-Prob of a Trajectory

- Note that gradients of everything that doesn't depend on θ is 0.

$$\nabla_{\theta} \log P(\tau|\theta) = \nabla_{\theta} \log \rho_0(s_0) + \sum_{t=0}^T (\nabla_{\theta} \log P(s_{t+1}|s_t, a_t) + \nabla_{\theta} \log \pi_{\theta}(a_t|s_t))$$

$$= \sum_{t=0}^T (\nabla_{\theta} \log \pi_{\theta}(a_t|s_t))$$

Fact #4

- Log-Derivative Trick:

- This is based on the rule from calculus that the derivative of $\log x$ is $1/x$

$$\nabla_{\theta} P(\tau|\pi) = P(\tau|\pi) \nabla_{\theta} \log P(\tau|\theta)$$

$$\frac{d}{dx} \log g(x) = \frac{1}{g(x)} \frac{d}{dx} g(x) \quad \Rightarrow \quad g(x) \frac{d}{dx} \log g(x) = \frac{d}{dx} g(x)$$

Derivation of Policy Gradient

$$\begin{aligned}\nabla_{\theta} J(\pi_{\theta}) &= \nabla_{\theta} E_{\tau \sim \pi_{\theta}} [R(\tau)] \\ &= \nabla_{\theta} \sum_{\tau} P(\tau|\theta) R(\tau) \\ &= \sum_{\tau} \nabla_{\theta} P(\tau|\theta) R(\tau) \\ &= \sum_{\tau} P(\tau|\theta) \nabla_{\theta} \log P(\tau|\theta) R(\tau) && \text{Fact \#4} \\ &= E_{\tau \sim \pi_{\theta}} [\nabla_{\theta} \log P(\tau|\theta) R(\tau)] \\ &= E_{\tau \sim \pi_{\theta}} [\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) R(\tau)] && \text{Fact \#3}\end{aligned}$$

The Policy Gradient

- We can now perform gradient ascent to improve our policy!

$$\theta_{k+1} \leftarrow \theta_k + \alpha \nabla_{\theta} J(\pi_{\theta}) \Big|_{\theta_k}$$

$$\nabla_{\theta} J(\pi_{\theta}) = E_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau) \right]$$

Estimate with a
sample mean over a
set D of policy rollouts
given current
parameters

$$\approx \frac{1}{|D|} \sum_{\tau \in D} \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau)$$

How would you implement this?

1. Start with random policy parameters θ_0
2. Run the policy in the environment to collect N rollouts (episodes) of length T and save returns of each trajectory.

$$a_t \sim \pi_{\theta}(\cdot | s_t) \Rightarrow (s_0, a_0, r_0, s_1, a_1, r_1, \dots, r_T, s_{T+1})$$

$$D = \{\tau_1, \dots, \tau_N\}, \quad R = \{R(\tau_1), \dots, R(\tau_N)\}$$

3. Compute policy gradient

$$\nabla_{\theta} J(\pi_{\theta}) = E_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau) \right]$$

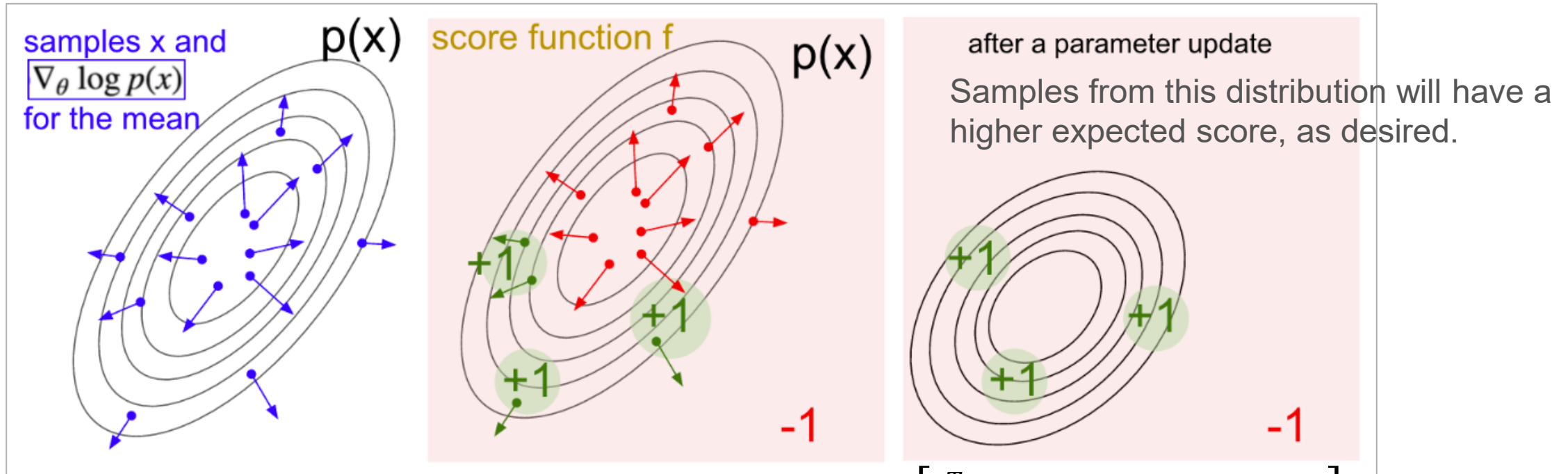
4. Update policy parameters

$$\theta_{k+1} \leftarrow \theta_k + \alpha \nabla_{\theta} J(\pi_{\theta}) \Big|_{\theta_k}$$

5. Repeat (Go to 2)

Some more intuition (thanks to Andrej Karpathy)

- Blue Dots: samples from Gaussian
- Blue arrows: gradients of the log probability with respect to the gaussian's mean parameter
- We score each sample
- Red have score -1
- Green have scores +1
- To update the Gaussian mean parameter, we average up all the green arrows, and the *negative* of the red arrows.



$$\nabla_{\theta} J(\pi_{\theta}) = E_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau) \right]$$

Policy Gradient RL Algorithms

- We can directly update the policy to achieve high reward.
- Pros:
 - Directly optimize what we care about: Utility!
 - Naturally handles continuous action spaces!
 - Can learn specific probabilities for taking actions.
 - Often more stable than value-based methods (e.g. DQN).
- Cons:
 - On-Policy -> Sample-inefficient we need to collect a large set of new trajectories every time the policy parameters change.
 - Q-Learning methods are usually more data efficient since they can reuse data from any policy (Off-Policy) and can update per sample.

Many forms of policy gradients

$s_0 a_0 r_1 s_1 a_1 r_2 s_2 \dots$ $\underline{s_t} a_t \xrightarrow{r_{t+1}}$

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \Phi_t \right]$$

$\sum$

What we derived: $\Phi_t = R(\tau),$

Follows a similar derivation:

$$\Phi_t = \sum_{t'=t}^T R(s_{t'}, a_{t'}, s_{t'+1}),$$

<https://medium.com/@thechrisyoon/deriving-policy-gradients-and-implementing-reinforce-f887949bd63>

- What is better about the second approach?
 - Focuses on rewards in the future!
 - Less variance -> less noisy gradients.

Many forms of policy gradients

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \Phi_t \right]$$

$$\Phi_t = \sum_{t'=t}^T R(s_{t'}, a_{t'}, s_{t'+1}),$$

Looks familiar....

$$\Phi_t = Q^{\pi_{\theta}}(s_t, a_t)$$

- Now we have an approach that combines a parameterized policy and a parameterized value function!

I rotate
the piece



Really bad
action

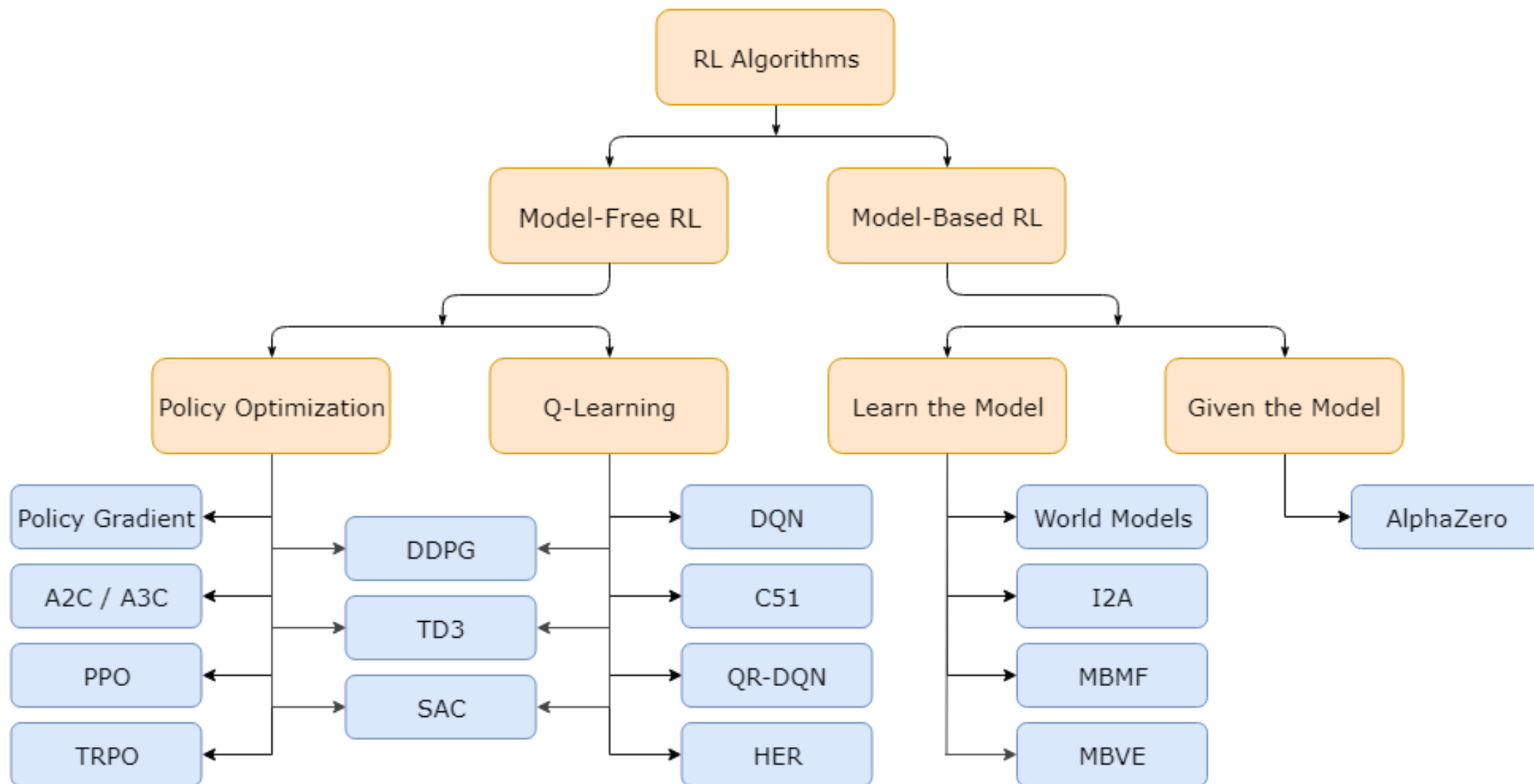


Actor



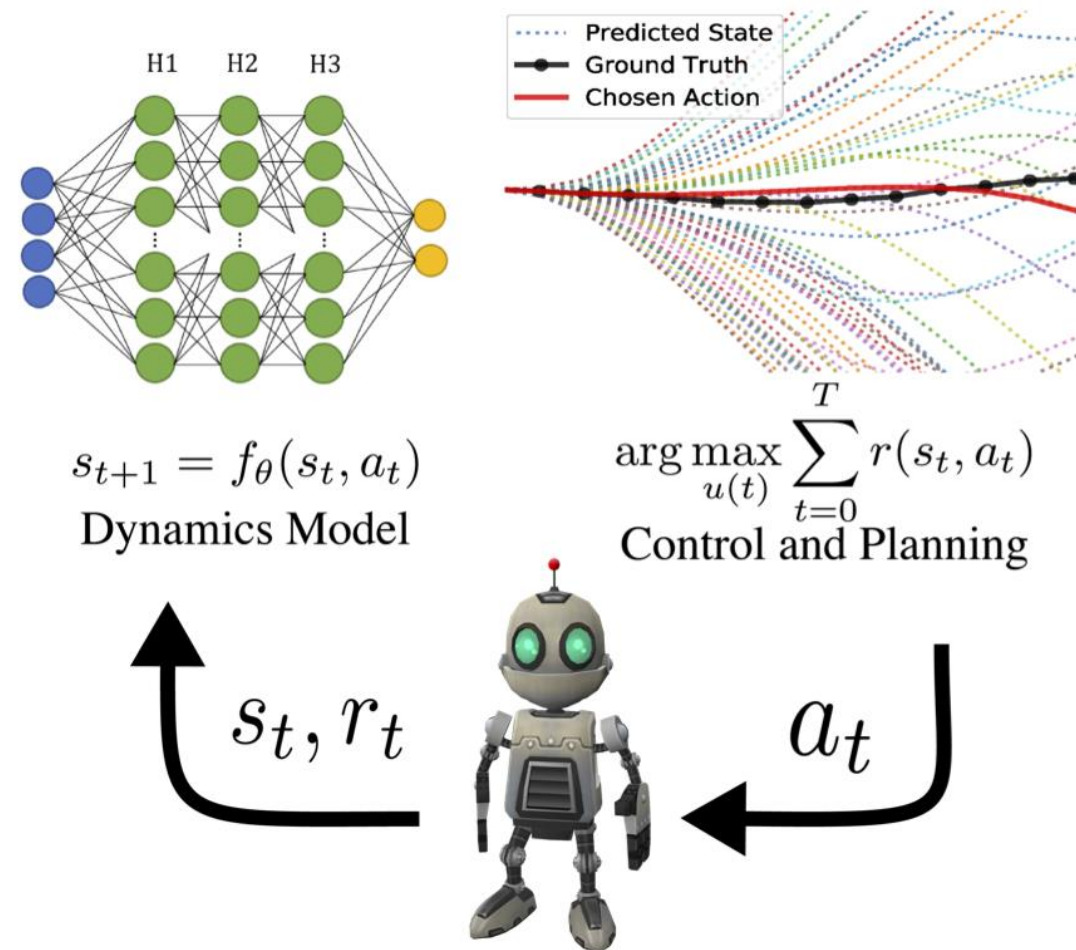
Critic

Rough Taxonomy of RL Algorithms



Model-Based RL via Model-Predictive Control

- Use model to plan good looking sequence of actions.
- Take a step
- Update model of transitions
- Repeat



When might RL be a good tool for your problem?

When might RL be a good tool for your problem?

- Is your problem a sequential decision making problem?
- Are there “actions” that effect the next “state”?
- Do you know the rules of these effects?
- Can you write down a clear objective/score/reward/cost?
- Do you have a simulator?
- Lots of examples of sequences of decisions and their long-term consequences?
- Is it unclear what to do in each state? Exploration required?
- Are you looking for unique/creative/super-human solutions?

When might RL not be a good tool?

When might RL not be a good tool?

- Single step or static problem
- No clear reward signal.
- Reward signal is unavailable or very hard to write down.
- Well-known model of the environment.
- Deterministic environment
- Low-tolerance for exploration and trial and error
- No need for adaptive or novel solutions. The goal is to perform the task in a very predictable way.

-
- Next time: Alpha Go